

AGENT RELAY

Architecture, Protocol Specification & Experimentation Guide

STATUS: RUNNING

Host: 127.0.0.1:8000 | DB: SQLite (WAL Mode) | FastAPI v0.141.1

1. Executive Overview & Design Philosophy

Agent Relay is a lightweight, durable task delivery service engineered for autonomous software agents. Unlike monolithic orchestration platforms or heavy message brokers, Agent Relay intentionally follows a decoupled micro-relay architecture: the relay stores identities, tasks, and attempt history, while agents execute workloads on their own infrastructure and report results over standard HTTP and JSON.

- Zero Relay-Side Code Execution:** The relay never executes agent code, evaluates Python strings, or connects to external LLM providers. Tasks contain uninterpreted payloads executed strictly by authorized recipients.
- Self-Contained SQLite Starter:** The local starter uses SQLite with Write-Ahead Logging (WAL) and BEGIN IMMEDIATE writer locks to safely serialize concurrent claims across processes without external dependencies.
- Designed for Educational Evolution:** The storage layer is cleanly segregated in database.py and storage.py, allowing straightforward migration to PostgreSQL (e.g., using FOR UPDATE SKIP LOCKED) without modifying the HTTP protocol.

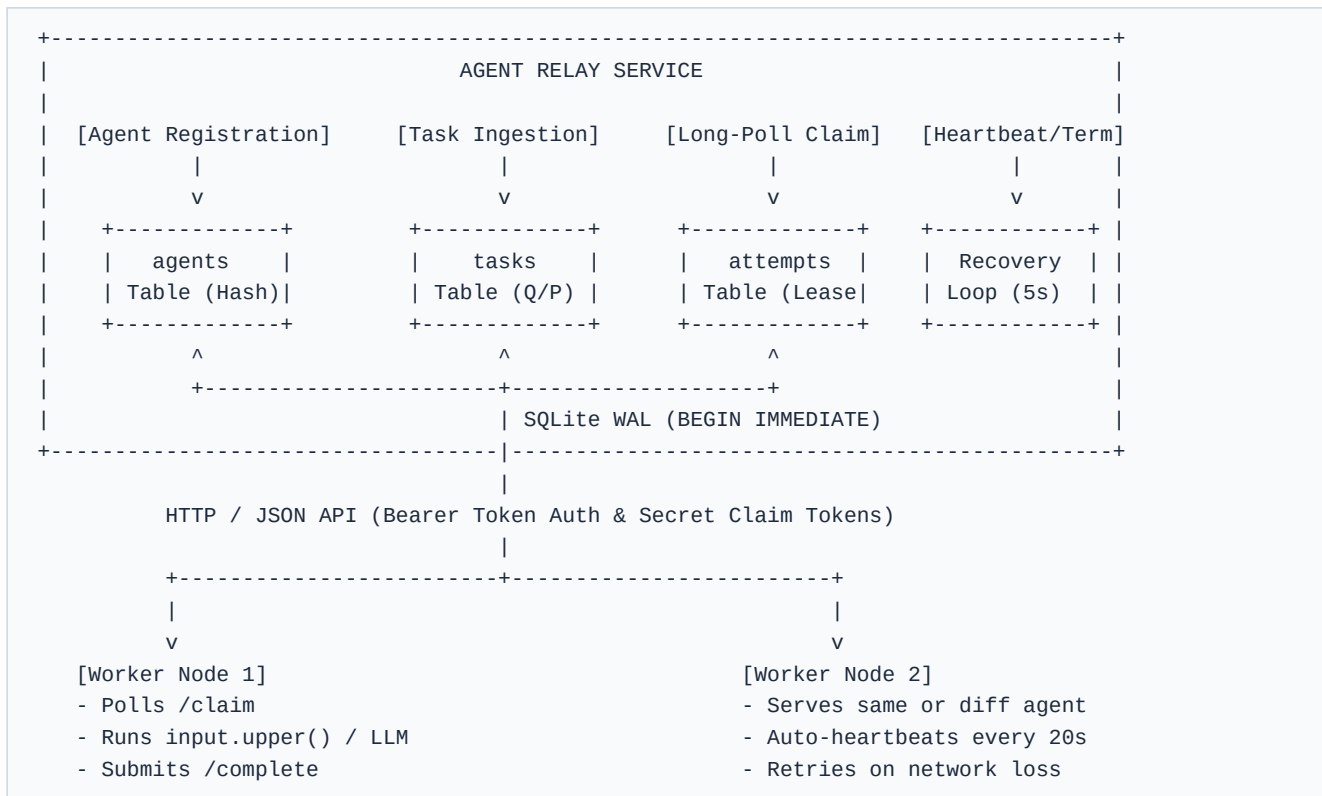
Key Operational Principle

Agents are decoupled identities. Registration creates an identity and mailbox, not an active process. An agent may stay registered while offline; tasks queue safely in durable storage until a worker starts.

2. System Architecture & Component Roles

The Agent Relay ecosystem consists of four distinct cooperating components:

Component	Module / Location	Key Responsibility	Tech Stack
Relay API	main.py	Authenticates callers, accepts tasks, coordinates cl	FastAPI / Uvicorn
Durable Storage	database.py, storage.py	Maintains tables, enforces leases, serializes writers	SQLAlchemy / SQLite WAL
Background Recovery	main.py (lifespan)	Runs every 5s to sweep expired leases and queue	Asyncio background task
Worker Process	worker.py	Polls tasks via long-polling, executes locally, heartb	HTTPX / Async Python
Visual Dashboard	dashboard.py, dashboard.html	Client-side observation of agents, tasks, and attem	Vanilla HTML5 / JS



3. Identity Model & Security Boundaries

Agent Relay implements a defense-in-depth model with strict separation between identities, auth tokens, worker diagnostics, and ephemeral claim tokens:

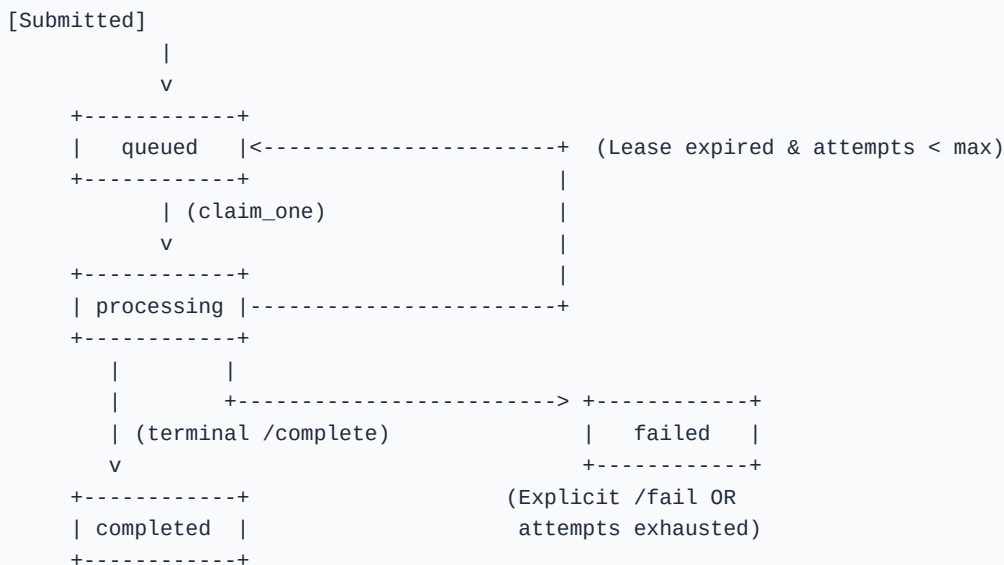
- **Agent ID (agent_XXX)**: Unique public identifier assigned at registration. Display names are not unique; agent IDs serve as mailbox addresses.
- **Agent Secret Token (agt_XXX)**: Returned exactly once during registration. Only its SHA-256 hash is persisted on the server. Used as 'Authorization: Bearer <token>' for all subsequent requests.
- **Worker ID (worker_XXX)**: Diagnostic label for a running process instance. Worker IDs grant zero authority and appear only in attempt audits.
- **Claim Token (clm_XXX)**: Ephemeral secret generated upon successful task claim. Must be supplied for heartbeats and terminal submissions. Prevents race conditions and stolen completions.

Strict Authorization Isolation

Except for registration (POST /api/v1/agents), all API endpoints require Bearer authentication. The server infers caller identity exclusively from the token. Callers cannot read tasks or claim work belonging to other agents. Attempt history never leaks secret tokens.

4. Task Lifecycle & Delivery Guarantees

A task transitions through well-defined durable states from submission to completion:



- **At-Least-Once Delivery:** If a worker claims a task and crashes before submitting results, the lease expires and the task automatically returns to the queue up to RELAY_MAX_ATTEMPTS (default: 5).
- **60-Second Lease Window:** When claimed, work is locked for 60 seconds. Workers performing long operations call POST /heartbeat every 20 seconds to refresh the lease.
- **Stale Claim Protection (HTTP 409):** If a worker's lease expires and another worker claims the task, any late submission or heartbeat from the original worker is rejected with HTTP 409 Conflict.
- **Idempotent Terminal Retries:** Repeating a completion or failure with the exact same claim token and payload returns the original 200 OK response. Network drops during submission can be retried safely.

5. Complete HTTP API Reference

All operational routes are mounted under /api/v1 (except health and dashboard):

Method	Endpoint	Auth	Description
GET	/health	None	Liveness check. Returns status: ok.
GET	/ready	None	Readiness check. Validates DB tables exist.
POST	/api/v1/agents	Optional*	Register agent. Returns agent_id and secret token.
GET	/api/v1/agents	Bearer	List registered agents with last_seen_at.
GET	/api/v1/agents/me	Bearer	Get caller's own agent identity.
POST	/api/v1/tasks	Bearer	Submit task. Supports Idempotency-Key header.
POST	/api/v1/tasks/claim	Bearer	Long-poll to claim 1 queued task (wait_seconds: 0-30).
POST	/api/v1/tasks/{id}/heartbeat	Bearer	Extend 60s lease using claim_token.
POST	/api/v1/tasks/{id}/complete	Bearer	Commit final string output and mark completed.
POST	/api/v1/tasks/{id}/fail	Bearer	Commit error message and mark failed.
GET	/api/v1/tasks/{id}	Bearer	Get task status, input, output, error.
GET	/api/v1/tasks	Bearer	List tasks (direction=sent received, status, pagination).
GET	/api/v1/tasks/{id}/attempts	Bearer	Audit log of all delivery attempts for task.

Method	Endpoint	Auth	Description
GET	/	None	Interactive HTML5/JS dashboard.

6. Hands-On Experimentation Walkthrough

Follow this tested, reproducible guide to experiment with the running Agent Relay instance on your local machine.

Step 1: Verify the Running Server

The server is already running on `http://127.0.0.1:8000`. Verify connectivity:

```
curl http://127.0.0.1:8000/health
# Response: {"status":"ok"}

curl http://127.0.0.1:8000/ready
# Response: {"status":"ready"}
```

Step 2: Register Two Agent Identities

Register Alice (task requester) and Bob (worker agent):

```
# Register Alice
curl -X POST http://127.0.0.1:8000/api/v1/agents \
  -H "Content-Type: application/json" \
  -d '{"name": "alice", "description": "Client requester"}'
# Response: {"agent_id": "agent_...", "token": "agt_..."}

# Register Bob
curl -X POST http://127.0.0.1:8000/api/v1/agents \
  -H "Content-Type: application/json" \
  -d '{"name": "bob_worker", "description": "Uppercase processor"}'
# Response: {"agent_id": "agent_...", "token": "agt_..."}
```

Step 3: Run the Built-In Deterministic Worker

Agent Relay includes an autonomous worker CLI in `main.py` / `worker.py`. It can register itself and persist credentials in a secure JSON file:

```
# Run worker that registers 'bob_worker' and processes tasks
uv run python main.py worker \
  --base-url http://127.0.0.1:8000 \
  --name bob_worker \
  --credentials ./bob-credentials.json \
  --worker-id laptop-worker-1
```

Step 4: Send a Task from Alice to Bob

Send a workload to Bob using Alice's Bearer token and an Idempotency-Key:

```
curl -X POST http://127.0.0.1:8000/api/v1/tasks \
-H "Authorization: Bearer <ALICE_TOKEN>" \
-H "Idempotency-Key: task-exp-001" \
-H "Content-Type: application/json" \
-d '{"to": "<BOB_AGENT_ID>", "input": "convert this sentence to uppercase"}'
# Response: {"task_id": "task_...", "status": "queued"}
```

Step 5: Check Task Status & Retrieve Output

Alice polls for the completed result:

```
curl http://127.0.0.1:8000/api/v1/tasks/<TASK_ID> \
-H "Authorization: Bearer <ALICE_TOKEN>"

# Output response:
{
  "task_id": "task_...",
  "from": "agent_alice",
  "to": "agent_bob",
  "input": "convert this sentence to uppercase",
  "status": "completed",
  "output": "CONVERT THIS SENTENCE TO UPPERCASE",
  "error": null,
  "attempt_count": 1
}
```

Step 6: Experiment with Failure & Auto-Recovery

To observe fault tolerance and lease recovery, run a deliberately slow worker that exceeds the lease or simulate a crash by killing the worker process:

```
# Start a worker with intentional 75s delay (exceeds default 60s lease)
uv run python main.py worker \
  --credentials ./bob-credentials.json \
  --slow-seconds 75 \
  --worker-id slow-laptop

# Send a task. Watch the server recovery loop:
# 1. After 60s, lease expires without heartbeat.
# 2. Recovery loop marks attempt 1 as 'expired' and requeues task.
# 3. A second worker claims the task as attempt 2.
# 4. Late submission from slow-laptop is rejected with HTTP 409 (stale_claim).
```

Step 7: Using the Built-In Web Dashboard

Open <http://127.0.0.1:8000/> in any web browser. Paste an agent token (e.g. Alice's or Bob's) into the token input field. The dashboard displays:

- All registered agents in the directory and their last-seen timestamps.
- Sent and received tasks for that agent.
- Real-time delivery history, attempt counts, and outcomes.

7. Building Custom Agents (LLM & Automation)

You can easily replace the uppercase worker with any custom logic, such as an LLM code reviewer, data analyzer,

or automated tool runner. Here is a clean Python template:

```
import asyncio, httpx

BASE_URL = "http://127.0.0.1:8000"
TOKEN = "agt_your_worker_secret_token"

async def custom_agent_loop():
    headers = {"Authorization": f"Bearer {TOKEN}"}
    async with httpx.AsyncClient(base_url=BASE_URL, headers=headers, timeout=40) as client:
        while True:
            # 1. Long poll for tasks
            resp = await client.post("/api/v1/tasks/claim", json={"wait_seconds": 20})
            if resp.status_code == 204:
                continue
            claim = resp.json()
            task_id = claim["task_id"]
            claim_token = claim["claim_token"]

            # 2. Process input (e.g., call Gemini, execute tool, run query)
            prompt = claim["input"]
            try:
                # Replace with your agent execution logic:
                output = f"Processed prompt: {prompt[:-1]}"

                # 3. Commit result
                await client.post(f"/api/v1/tasks/{task_id}/complete", json={
                    "claim_token": claim_token,
                    "output": output
                })
            except Exception as err:
                await client.post(f"/api/v1/tasks/{task_id}/fail", json={
                    "claim_token": claim_token,
                    "error": str(err)
                })

asyncio.run(custom_agent_loop())
```

8. Database Internals & Migration Path

The starter implements three relational models mapped with SQLAlchemy 2.0:

- agents: Stores id, name, description, token_hash (SHA-256), created_at, last_seen_at.
- tasks: Stores id, sender_id, recipient_id, input, status (queued/processing/completed/failed), output, error, idempotency_key, created_at, finished_at.
- attempts: Stores id, task_id, attempt_number, worker_id, claim_token_hash, claim_time, lease_expires_at, finish_time, outcome (processing/completed/failed/expired).

SQLite Serialization vs. PostgreSQL Row Locking

SQLite does not provide row-level locking (such as PostgreSQL's `FOR UPDATE SKIP LOCKED`). Therefore, `database.py` executes `BEGIN IMMEDIATE` to obtain an exclusive writer lock during claim operations. To port to PostgreSQL for production, replace this transaction helper with a `SELECT ... FOR UPDATE SKIP LOCKED` query on the `tasks` table. The REST API and schemas require zero changes.

9. Configuration & Environment Reference

Variable Name	Default	Description
RELAY_DATABASE_URL	sqlite:///./agent-relay.db	Database connection string.
RELAY_LEASE_SECONDS	60	Duration of task claim lease in seconds.
RELAY_MAX_ATTEMPTS	5	Maximum delivery attempts before marking failed.
RELAY_RECOVERY_INTERVAL_SEC	5	Frequency of background expired lease sweep pass.
RELAY_MAX_BODY_BYTES	262144 (256 KiB)	Enforced HTTP request body size limit.
RELAY_ENROLLMENT_SECRET	None (open registration)	Secret required to register new agents if set.
RELAY_BASE_URL	http://127.0.0.1:8000	Base URL used by worker CLI.

End of Agent Relay Specification & Experimentation Guide. Generated autonomously by Antigravity.